

Computer Programming for Mathematics I

040223101 — Chapter 1: Introduction to Computers and Python

Department of Mathematics, Faculty of Applied Science
King Mongkut's University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul | Semester 1, Academic Year
2569



Chapter 1 Learning Objectives

Upon completing this chapter, students will be able to:

O1

Computer System Components

Explain the main components of a computer system – both hardware and software.

O2

IPO Model

Describe the Input - Process - Output (IPO) data processing concept.

O3

High-Level vs. Machine Language

Explain the difference between high-level languages and machine language, and the role of translators.

O4

Python Environment Setup

Install and launch a Python programming environment.

O5

First Python Program

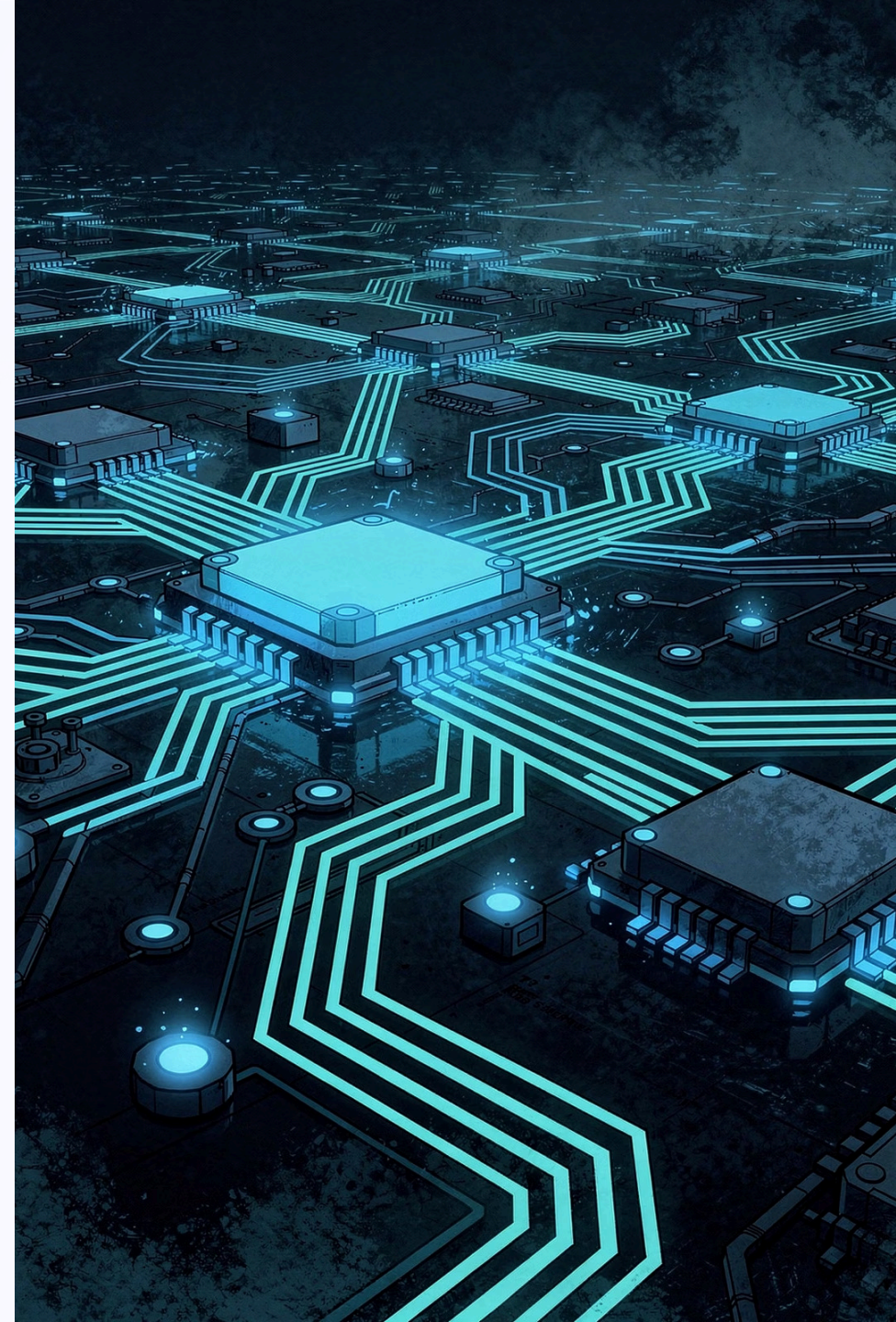
Write and run a first Python program using the `print()` command.

 This chapter aligns with Course Learning Outcome **CLO 1**.

Chapter 1: Introduction to Computers and Python

Computers are essential tools for solving mathematical and scientific problems today. To instruct a computer to perform a desired task, we must write a **program** – an ordered set of step-by-step instructions.

This chapter lays the foundational understanding of how computers work, the concept of data processing, and introduces the **Python programming language**, which will be used throughout this entire course.



1.1 Components of a Computer System

A computer system consists of two major parts that work together:

Hardware

The physical, tangible parts of the computer that you can touch and see – such as the processor, memory chips, keyboard, and monitor.

Software

The set of instructions or programs that direct the hardware to perform specific tasks – from the operating system to the applications we write ourselves.

Understanding how hardware and software interact is the first step toward writing effective programs. Every program we write ultimately becomes instructions that the hardware executes.

1.1.1 Hardware: The Four Functional Units

Computer hardware is divided into four functional units based on their roles:

Input Unit

Receives data and commands from the user. Examples: keyboard, mouse, touchscreen.

CPU

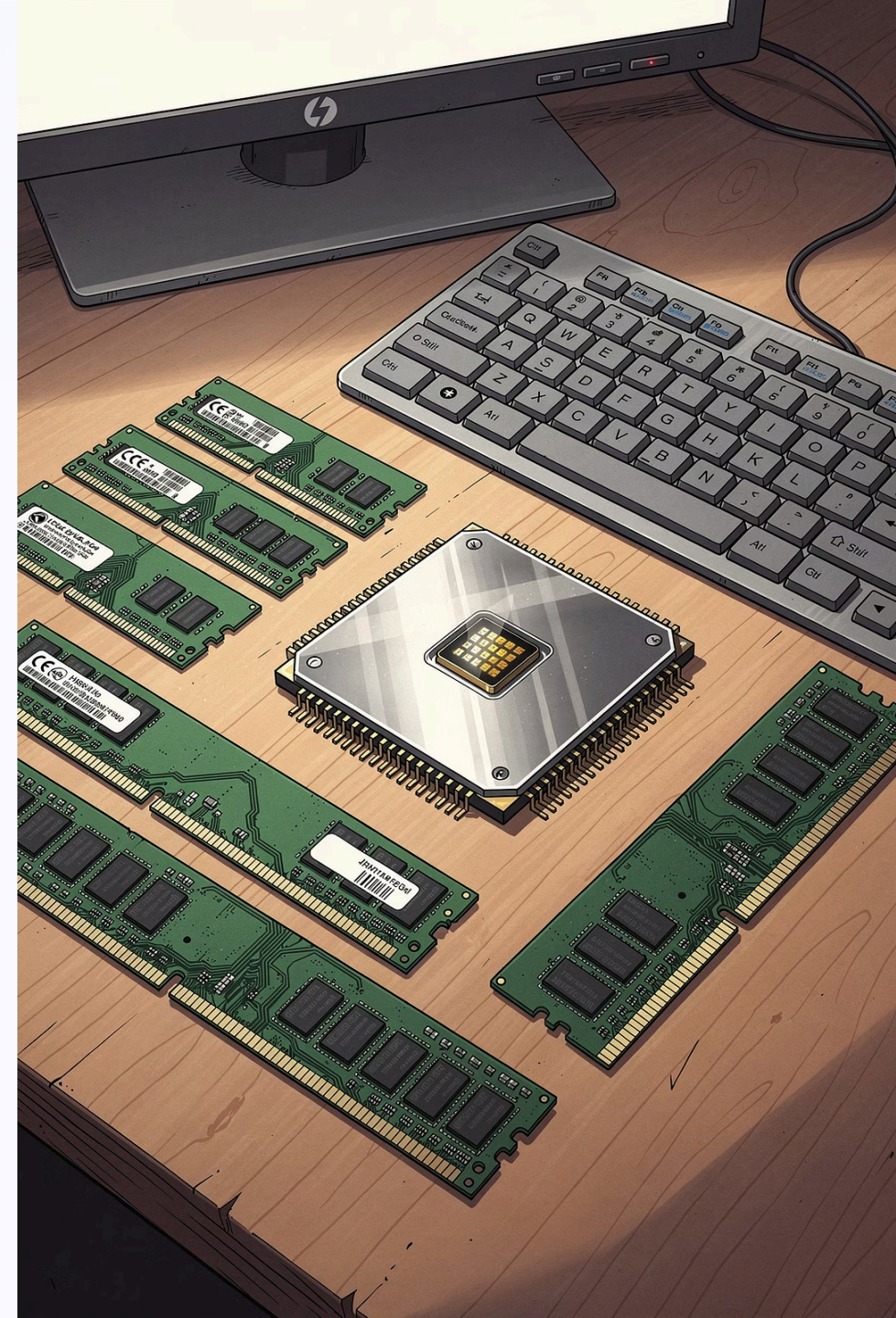
The "brain" of the computer. Contains the **Control Unit** (manages execution order) and the **ALU** (performs arithmetic and logical operations).

Memory Unit

RAM: stores data and programs during execution (volatile). **HDD/SSD:** stores data permanently.

Output Unit

Displays results to the user. Examples: monitor, printer, speakers.



Inside the CPU: Control Unit & ALU

Control Unit (CU)

Manages and coordinates the sequence of operations within the CPU. It fetches instructions from memory, decodes them, and directs the ALU and other components to execute them in the correct order.

Arithmetic Logic Unit (ALU)

Performs all mathematical calculations – addition, subtraction, multiplication, division – as well as logical comparisons such as greater than, less than, and equal to. Every computation in a program ultimately passes through the ALU.

❏ **Key Insight:** The CPU works closely with RAM. Programs and data must be loaded into RAM before the CPU can process them. Therefore, RAM speed directly affects overall system performance.

Memory: RAM vs. Secondary Storage

Primary Memory – RAM (Random Access Memory)

Stores the currently running programs and data. The CPU accesses RAM directly and very quickly. However, RAM is **volatile** – all data is lost when the computer is powered off. The more RAM a system has, the more programs it can run simultaneously without slowing down.

Secondary Storage – HDD / SSD

Stores data **permanently**, even when the power is off. Hard Disk Drives (HDD) use spinning magnetic platters, while Solid State Drives (SSD) use flash memory and are significantly faster. Your Python source files (.py) are saved here.

1.1.2 Software: System vs. Application



System Software

Manages the computer's hardware resources and provides a platform for other software to run. The most important example is the **Operating System (OS)** – such as Windows, macOS, or Linux – which handles file management, memory allocation, and device communication.



Application Software

Programs designed to perform specific tasks for the user. Examples include web browsers, spreadsheet calculators, and – most relevantly for this course – the **Python programs we will write ourselves** to solve mathematical problems.

1.2 Data and Information Processing

Data vs. Information

Data refers to raw, uninterpreted facts. For example, the numbers 25, 30, 28 on their own carry no meaning without context.

Information is the result of processing data to give it meaning. For example: *"The average temperature is 27.67 degrees Celsius"* – this is information derived from the raw data above.

The transformation from data to information is the core purpose of most computer programs.

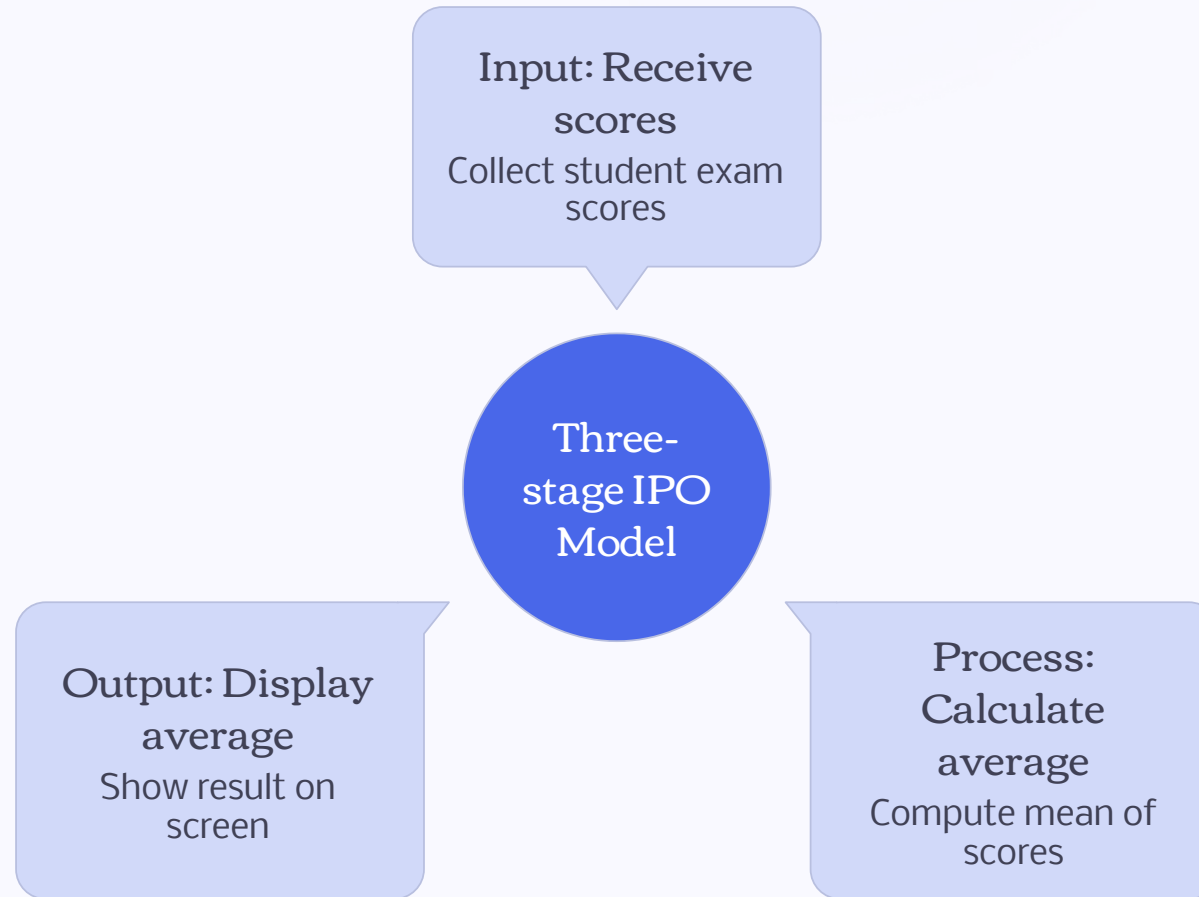
Why This Matters

Every program we write in this course will follow this pattern: we start with raw input data, apply some mathematical or logical process, and produce meaningful output information.

Understanding this distinction helps us design better programs – we always ask: *What data do I need? What processing is required? What information should I produce?*

The IPO Model: Input-Process-Output

Most computer programs can be described using the **IPO model**, which breaks program execution into three stages:



This model is a powerful mental framework. Before writing any program, ask yourself: *What is my input? What processing must occur? What output do I need?* Answering these three questions defines the structure of your program.

IPO Model – Detailed Example

Stage	Meaning	Example
Input (รับเข้า)	Receive data into the program	Receive student exam scores
Process (ประมวลผล)	Calculate or transform data according to logic	Find the average of the scores
Output (แสดงผล)	Display the result	Show the average score on screen

- ✔ The IPO model applies to virtually every program you will write – from simple calculations to complex mathematical algorithms. Always plan your IPO before coding.



1.3 Programming Languages

Computers understand only **machine language** – binary code consisting of 0s and 1s. Writing programs directly in binary is extremely difficult and error-prone for humans.

Instead, we write in **high-level languages** that resemble human language and mathematical notation. A **language translator** then converts our code into machine language that the CPU can execute.

Compiler vs. Interpreter

Compiler

Translates the **entire program** into machine language before execution. The compiled output is a standalone executable file. Examples: C, C++, Fortran. Advantage: faster execution after compilation. Disadvantage: must recompile after every change.

Interpreter

Translates and executes the program **one line at a time**. No separate compilation step is needed. Examples: Python, JavaScript. Advantage: immediate feedback, easier to debug interactively. Disadvantage: slightly slower execution.

 **Python uses an interpreter.** This means you can test code line by line in interactive mode and see results immediately – a major advantage for learning and mathematical exploration.

Why Python for Mathematics?

Python is a high-level, interpreter-based language with outstanding strengths for beginners and mathematical work:

Readable Syntax

Python's syntax is concise and close to English and mathematical notation, making it easy to read, write, and understand – even for first-time programmers.

Free & Open-Source

Python is completely free to use and works on all major operating systems: Windows, macOS, and Linux. No licensing costs.

Rich Math Libraries

Python has a vast ecosystem of scientific libraries: `math`, `NumPy`, `SymPy`, `Matplotlib`, and many more – all purpose-built for mathematical and scientific computing.



Python's Scientific Library Ecosystem



math

Python's built-in standard library for mathematical functions: square roots, trigonometry, logarithms, constants like π and e . No installation required.



NumPy

Numerical Python – provides powerful array and matrix operations, linear algebra, Fourier transforms, and random number generation at high speed.



SymPy

Symbolic mathematics library – performs algebraic manipulation, calculus (derivatives, integrals), equation solving, and more, with exact symbolic results.



Matplotlib

The standard Python plotting library – creates 2D graphs, charts, and visualizations of mathematical functions and data sets.

1.4 Installing and Running Python

Students can start writing Python programs through several channels. Choose whichever suits your situation:

1

Python + IDLE

Install Python from **python.org**. The IDLE editor is included automatically and provides a simple environment for writing and running programs.

2

Anaconda + Jupyter

Install the **Anaconda** distribution, which comes pre-packaged with scientific libraries and the **Jupyter Notebook** interface – ideal for mathematical work.

3

Google Colab

Use **Google Colab** (colab.research.google.com) – runs entirely in a web browser with no installation required. Best for getting started quickly.

Two Modes of Running Python

Interactive Mode

Type commands one line at a time and see results immediately. Identified by the `>>>` prompt. Perfect for quick calculations, testing expressions, and exploring Python features.

Example session:

```
>>> 2 + 3 * 4
14
>>> 10 / 4
2.5
>>> "Hello"
'Hello'
```

Script Mode

Write multiple lines of code and save them in a file with the `.py` extension. Then run the entire file at once. This is the standard approach for writing complete programs.

Script mode is essential for any program longer than a few lines, and is what you will use for all assignments and projects in this course.



Interactive Mode – Live Examples

In interactive mode, Python evaluates each expression immediately after you press Enter. This makes it an excellent calculator and learning tool:

```
>>> 2 + 3 * 4
14
>>> 10 / 4
2.5
>>> "Hello"
'Hello'
```

Notice that Python follows standard mathematical operator precedence: multiplication (*) is evaluated before addition (+), so `2 + 3 * 4` equals `2 + 12 = 14`, not 20.

1.5 Your First Program: The print() Command

The `print()` function displays text or values on the screen. It is the very first command that beginners learn to write. Text strings must be enclosed in quotation marks – either single `' '` or double `" "`.

Source File: Olhello.py

```
print("Hello, World!")  
print("ยินดีต้อนรับสู่การเขียนโปรแกรม")
```

Output:

```
Hello, World!  
ยินดีต้อนรับสู่การเขียนโปรแกรม
```

Key Rules

- Text (strings) must be wrapped in quotation marks
- Both single `'...'` and double `"..."` quotes are valid
- Each `print()` call outputs on a new line by default
- The function name is lowercase: `print`, not `Print`

print() for Mathematical Expressions

The `print()` function can also evaluate and display mathematical expressions directly. The key distinction is whether you use quotation marks or not:

Source File: Olcalc.py

```
print(3 + 5)
print("3 + 5")
print(2 ** 10)
```

Output:

```
8
3 + 5
1024
```

Explanation

- `print(3 + 5)` – no quotes, so Python **calculates** the expression and prints 8
- `print("3 + 5")` – with quotes, Python treats it as **text** and prints literally 3 + 5
- `print(2 ** 10)` – the `**` operator means **exponentiation**, so $2^{10} = 1024$

 The `**` operator is Python's exponentiation operator. `2 ** 10` means 2 raised to the power of 10, which equals 1024.

Strings vs. Expressions in print()

With Quotation Marks → String

Python treats the content as literal text and prints it exactly as written. No calculation is performed.

```
print("2 ** 10")  
# Output: 2 ** 10
```

Without Quotation Marks → Expression

Python evaluates the mathematical expression and prints the computed result.

```
print(2 ** 10)  
# Output: 1024
```

This distinction is fundamental in Python. Whenever you want Python to *compute* something, omit the quotes. Whenever you want to display text *as-is*, use quotes. Confusing these two is one of the most common beginner mistakes.

1.6 Comments in Python

A **comment** is explanatory text in the source code that Python does *not* execute. Comments begin with the `#` symbol. Everything after `#` on that line is ignored by the interpreter.

Comments serve a vital purpose: they help **you and others understand the code** – what it does, why certain decisions were made, and how it works.

```
# Program to calculate the area of a rectangle
width = 4 # width of the rectangle
height = 5 # height of the rectangle
print(width * height) # display the area
```

Output:

```
20
```

- ✔ Good programmers write comments generously. Even if you understand your own code today, you may not remember your reasoning weeks later. Comments are a professional habit worth building from day one.

Comment Best Practices

→ Describe the Program's Purpose

Place a comment at the top of every file explaining what the program does. This is the first thing any reader will see.

→ Label Variables with Units or Meaning

When a variable name alone isn't clear enough, a short comment explaining its meaning or units (e.g., # in meters) prevents confusion.

→ Explain Non-Obvious Logic

Add inline comments next to lines that perform complex or non-obvious operations. Don't comment on things that are self-evident.

→ Keep Comments Up to Date

When you modify code, update the corresponding comments. Outdated comments are worse than no comments – they actively mislead readers.

1.6 Common Beginner Errors: SyntaxError

If you write a command with incorrect syntax, Python will report a **SyntaxError**. This is the most common type of error for beginners. Python cannot execute the program until all syntax errors are fixed.

Incorrect Code

```
print("Hello) # missing closing quote
```

Error Message

```
SyntaxError: unterminated string literal
```

How to Read Error Messages

Python's error messages tell you:

- **Error type:** e.g., `SyntaxError`
- **Description:** what went wrong (e.g., "unterminated string literal")
- **Line number:** where in the file the error occurred

Always read the error message carefully before trying to fix the code. It points directly to the problem location.

Common SyntaxError Causes

Missing Closing Quotation Mark

Every string that opens with " or ' must close with the same character. Forgetting the closing quote causes an "unterminated string literal" error.

```
print("Hello) # WRONG  
print("Hello") # CORRECT
```

Missing Closing Parenthesis

The print() function requires both an opening and a closing parenthesis. Missing the closing) causes a syntax error.

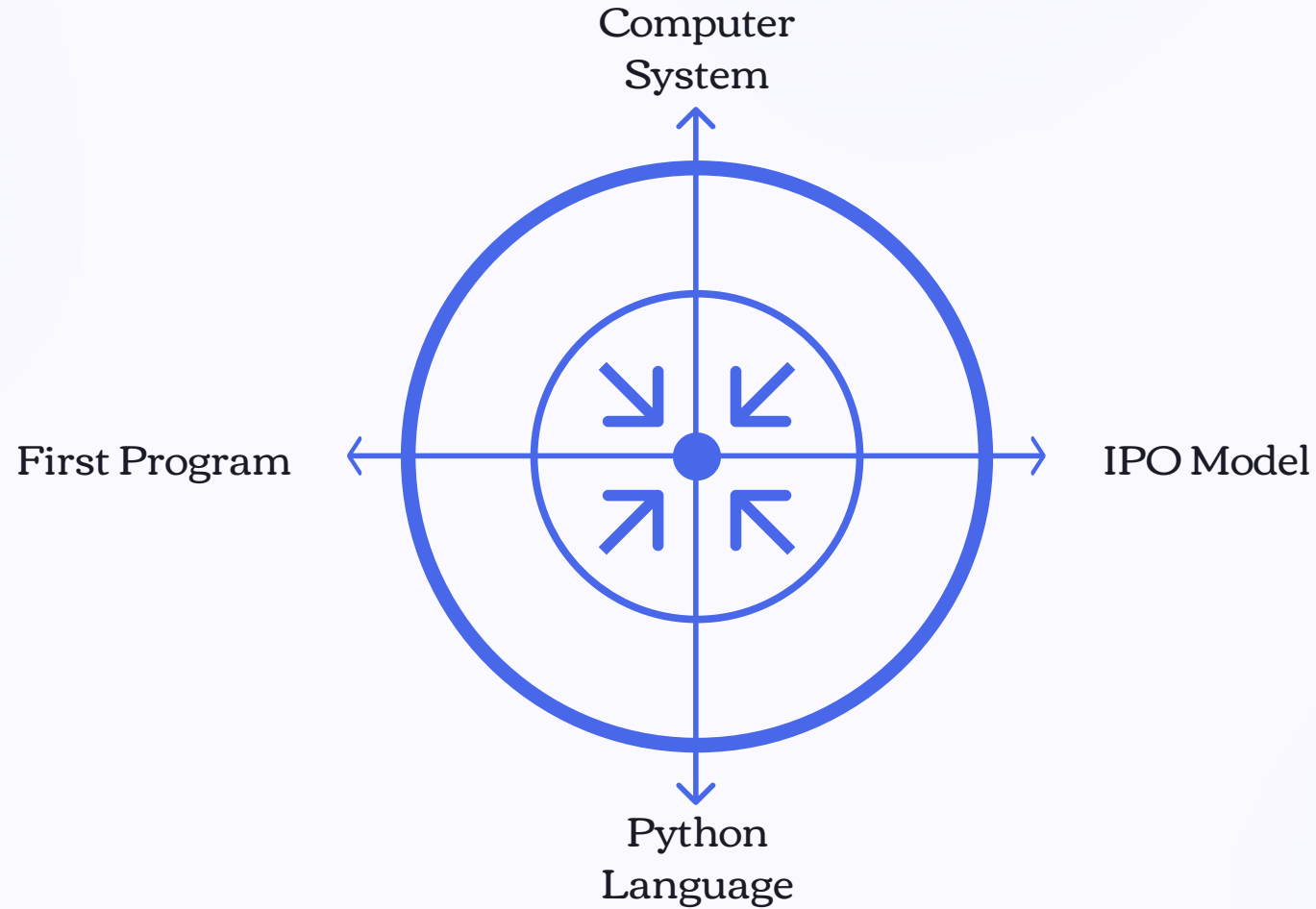
```
print("Hello" # WRONG  
print("Hello") # CORRECT
```

Incorrect Capitalization

Python is case-sensitive. Print() and PRINT() are not the same as print(). Always use lowercase for built-in functions.

```
Print("Hello") # WRONG  
print("Hello") # CORRECT
```

1.7 Chapter Summary



This chapter established the foundational knowledge needed for the rest of the course: how computers are structured, how data flows through programs, why Python is an ideal language for mathematics, and how to write and run your very first Python program.

Summary: Key Concepts at a Glance

Computer System

Hardware (Input, CPU, Memory, Output) + Software (System & Application) working together.

IPO Model

All programs follow Input → Process → Output. Plan these three stages before writing any code.

Python

High-level, interpreter-based, free, readable syntax, rich math libraries. Ideal for beginners and scientists.

print()

The fundamental output command. Use quotes for strings, no quotes for expressions. `#` starts a comment.

 **Next Chapter:** Variables and Input/Output – storing values, receiving user input, and displaying formatted results.

End-of-Chapter Exercises

The following exercises test your understanding of Chapter 1. Attempt each one before checking the solutions.

O1

Exercise 1 – Hardware Units

Describe the function of each of the four hardware units in a computer system, and give an example device for each unit.

O2

Exercise 2 – Data vs. Information

Explain the difference between "data" and "information" and provide an example from everyday life.

O3

Exercise 3 – Display Your Name

Write a program that displays the message "My name is ..." (insert your own name) on the screen.

O4

Exercise 4 – Mathematical Expression

Write a program that calculates and displays the result of $15 \times 8 - 100 \div 4$ using a single `print()` statement.

O5

Exercise 5 – Exponentiation

Write a program that displays the result of **2 raised to the power of 16**.

Exercise 1 – Discussion: The Four Hardware Units

Unit	Function	Example Devices
Input	Receives data and commands from the user	Keyboard, mouse, touchscreen, microphone
CPU	Processes instructions; contains CU and ALU	Intel Core i7, AMD Ryzen, Apple M-series
Memory	Stores programs and data (RAM = temporary; HDD/SSD = permanent)	DDR5 RAM, NVMe SSD, HDD
Output	Displays results to the user	Monitor, printer, speakers, projector

Exercise 2 – Discussion: Data vs. Information

Data (Raw Facts)

Unprocessed, uninterpreted facts with no inherent meaning on their own.

Example: The numbers 85, 92, 78, 90, 88 – these are just numbers without context.

Information (Processed Meaning)

Data that has been processed and given context, making it useful for decision-making.

Example: "The average exam score of the five students is **86.6 out of 100**, which is a B grade." This is information derived from the raw data.

Everyday example: Raw temperature readings (25°C, 30°C, 28°C) become information when processed: "This week's average temperature was 27.67°C, which is 2°C above the seasonal norm."

Exercise 3 – Solution: Display Your Name

Task: Write a program that displays "My name is ..." on the screen.

Source Code

```
print("My name is Somchai")
```

Output

```
My name is Somchai
```

Explanation

The text "My name is Somchai" is a **string literal** enclosed in double quotation marks. The `print()` function outputs it exactly as written to the screen.

Replace `Somchai` with your own name. The quotation marks must remain – they tell Python this is text, not a variable or expression.

Exercise 4 – Solution: Mathematical Expression

Task: Calculate and display $15 \times 8 - 100 \div 4$ using a single `print()` statement.

Source Code

```
print(15 * 8 - 100 / 4)
```

Output

```
95.0
```

Step-by-Step Calculation

Python follows standard mathematical operator precedence (PEMDAS/BODMAS):

1. $15 * 8 = 120$ (multiplication first)
2. $100 / 4 = 25.0$ (division next)
3. $120 - 25.0 = 95.0$ (subtraction last)

Note: The result is `95.0` (a float) because division (`/`) in Python always returns a floating-point number.

Exercise 5 – Solution: Exponentiation

Task: Write a program that displays the result of 2 raised to the power of 16.

Source Code

```
print(2 ** 16)
```

Output

```
65536
```

Explanation

The `**` operator in Python performs **exponentiation**. So `2 ** 16` means 2^{16} .

Calculating step by step:

- $2^1 = 2$
- $2^4 = 16$
- $2^8 = 256$
- $2^{16} = \mathbf{65,536}$

This result is significant in computing – 65,536 is the number of values representable by a 16-bit unsigned integer.

All Exercise Solutions at a Glance

Ex.	Code	Output
3	<pre>print("My name is Somchai")</pre>	My name is Somchai
4	<pre>print(15 * 8 - 100 / 4)</pre>	95.0
5	<pre>print(2 ** 16)</pre>	65536

- ❏ Exercises 1 and 2 are written-answer questions with no single correct code solution. Refer to the discussion slides for guidance on those answers.

Python Arithmetic Operators Reference

The following operators are used in Python for mathematical calculations. You will use all of these throughout the course:

Operator	Meaning	Example	Result
<code>+</code>	Addition	<code>3 + 5</code>	<code>8</code>
<code>-</code>	Subtraction	<code>10 - 4</code>	<code>6</code>
<code>*</code>	Multiplication	<code>6 * 7</code>	<code>42</code>
<code>/</code>	Division (float result)	<code>10 / 4</code>	<code>2.5</code>
<code>//</code>	Floor division (integer result)	<code>10 // 4</code>	<code>2</code>
<code>%</code>	Modulo (remainder)	<code>10 % 4</code>	<code>2</code>
<code>**</code>	Exponentiation	<code>2 ** 10</code>	<code>1024</code>

Python vs. Other Languages: A Quick Comparison

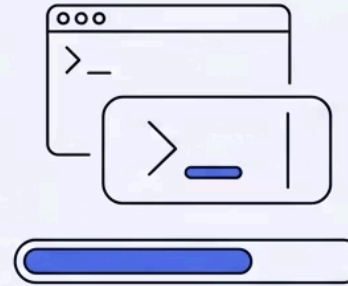
SYNTAX READABILITY

`var = 12` → Python: **VERY EASY**
(like English)

`var = c++;` → C++: **COMPLEX**
(verbose, symbols)

`var = 123;` → Java: **MODERATE**
(verbose, rigid)

SETUP REQUIRED



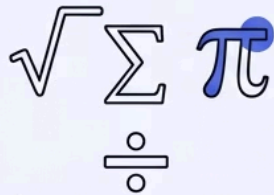
Python: **MINIMAL**
(easy install)

C++: **MODERATE**
(compilers)

Java: **MODERATE**
(JDK install)

MATH LIBRARIES

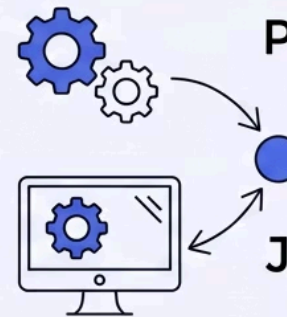
Python: **EXTENSIVE BUILT-IN**



C++: **LIMITED**
(need external libs)

Java: **MODERATE**
(standard libs)

INTERPRETER/COMPILER



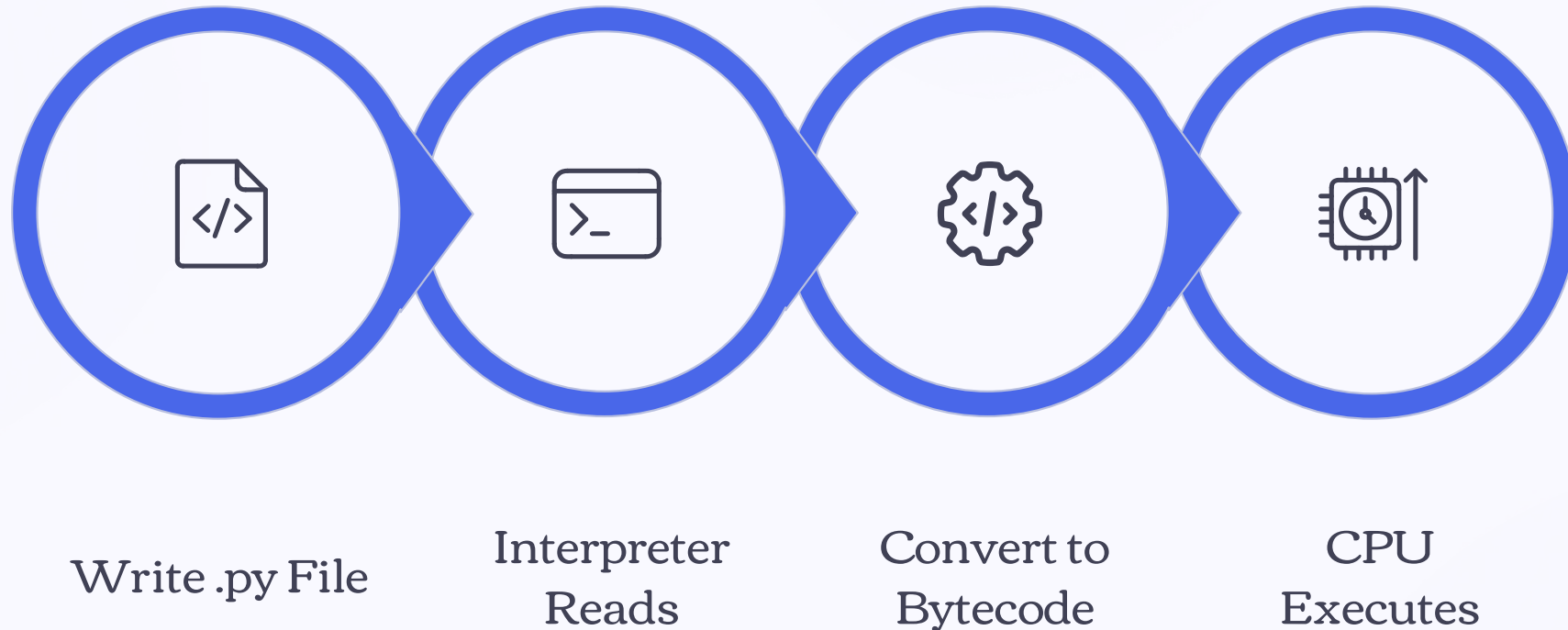
Python: **INTERPRETER**
(runs code directly)

C++: **COMPILER**
(makes executable)

Java: **COMPILER+JVM**
(platform independent)

Python stands out for its minimal setup, readable syntax, and rich mathematical library ecosystem – making it the preferred choice for mathematics education and scientific computing worldwide.

The Journey from Code to Execution



When you run a Python program, the interpreter reads your source code line by line, translates each line into low-level bytecode, and the CPU executes it. This is why Python gives you immediate feedback – errors are caught and reported as soon as the interpreter reaches the problematic line, rather than after a full compilation step.

Key Takeaways from Chapter 1

- Computers = Hardware + Software
The four hardware units (Input, CPU, Memory, Output) work together under the direction of software to process information.
- IPO Is the Universal Program Model
Every program receives input, processes it, and produces output. Always plan these three stages before writing code.
- Python Is Ideal for Math
Readable syntax, free to use, interpreter-based for instant feedback, and a rich ecosystem of scientific libraries.
- `print()` and `#` Are Your First Tools
Use `print()` to display output and `#` to write comments. These two tools alone let you write meaningful programs.



What's Coming Next: Chapter 2 Preview

In Chapter 2, we will build on this foundation and explore:



Variables

How to store and name values in memory so they can be reused throughout a program.



User Input

How to receive data from the user at runtime using the `input()` function.



Data Types

Integers, floats, and strings – and how Python handles each type differently.



Formatted Output

How to display results in a clean, readable format using f-strings and format specifiers.

Ready to Start Programming!

You now have the foundational knowledge to begin your journey in computer programming for mathematics. You understand how computers work, why Python is the right tool for this course, and how to write and run your very first programs.

Practice This Week

Complete all five end-of-chapter exercises. Experiment in interactive mode – try different arithmetic expressions and observe Python's behavior.

Set Up Your Environment

Install Python + IDLE, Anaconda, or create a Google Colab account. Make sure you can run a `print("Hello, World!")` program successfully before the next class.

Read Ahead

Preview Chapter 2 on variables and input/output to prepare for the next lecture session.